

CO3 AT 1

1.Code :

```
public class SynchronizedCounterDemo {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized void decrement() {
        count = count - 1;
    }

    public static void main(String[] args) throws InterruptedException {
        SynchronizedCounterDemo demo = new SynchronizedCounterDemo();

        Runnable task = () -> {
            for (int i = 0; i < 1000; i++) {
                demo.increment();
            }
        };

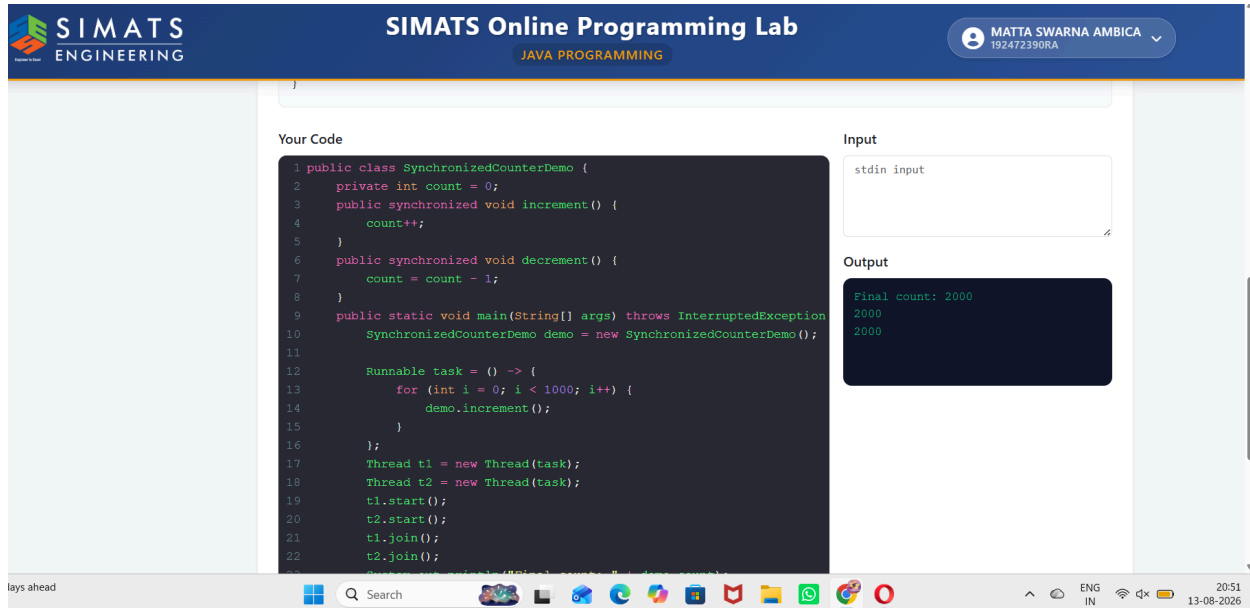
        Thread t1 = new Thread(task);
        Thread t2 = new Thread(task);
        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final count: " + demo.count);

        synchronized (demo) {
            System.out.println(demo.count);
        }

        int finalCount = demo.count;
        System.out.println(finalCount);
    }
}
```



1. Code :

```
import java.util.concurrent.*;
```

```
public class RunnableCallableDemo {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);
```

```
// Callable can return a value and throw exceptions
```

```
Callable<Integer> callableTask = () -> {
    return 10 / 0;
};
```

```
Future<Integer> future = executor.submit(callableTask);
```

```
try {
    // future.get() will throw an ExecutionException because of the division by zero
    int result = future.get();
    System.out.println(result);
} catch (ExecutionException e) {
    System.out.println("Exception occurred in callable task: " + e.getCause());
}
```

```
// Runnable cannot return a value; its run() method is void
```

```
Runnable r = () -> {
    int x = 5;
    System.out.println("Runnable executed, value: " + x);
};
```

```
executor.submit(r);
```

```
executor.shutdown();
```

```
}  
}
```

The screenshot shows the SIMATS Online Programming Lab interface. The header includes the SIMATS ENGINEERING logo, the lab name, and a user profile for MATTIA SWARNA AMBICA. The main area is divided into three sections: 'Your Code' with a Java code editor, 'Input' with a text field, and 'Output' with a console window.

```
1 import java.util.concurrent.*;  
2 public class RunnableCallableDemo {  
3     public static void main(String[] args) throws Exception {  
4         ExecutorService executor = Executors.newFixedThreadPool(2);  
5         Callable<Integer> callableTask = () -> {  
6             return 10 / 0;  
7         };  
8         Future<Integer> future = executor.submit(callableTask);  
9         try {  
10            int result = future.get();  
11            System.out.println(result);  
12        } catch (ExecutionException e) {  
13            System.out.println("Exception occurred in callable task: " + e.getMessage());  
14        }  
15        Runnable r = () -> {  
16            int x = 5;  
17            System.out.println("Runnable executed, value: " + x);  
18        };  
19        executor.submit(r);  
20        executor.shutdown();  
21    }  
22 }
```

Input: stdin input

Output: Exception occurred in callable task: java.lang.ArithmeticException: / by zero
Runnable executed, value: 5

2. Code :

```
import java.io.*;
```

```
public class TryWithResourcesDemo {  
    public static void main(String[] args) {  
        // Fix 1: Declare the resource inside the try-with-resources parentheses  
        try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {  
            String line = reader.readLine();  
            System.out.println(line);  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
  
        try (FileWriter writer = new FileWriter("out.txt")) {  
            writer.write("Hello");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
  
        // Fix 2: Class must implement AutoCloseable to be used in try-with-resources  
        class NotCloseable implements AutoCloseable {  
            void write() { }  
        }  
    }  
}
```

```

@Override
public void close() {
    // Required by AutoCloseable interface
}

try (NotCloseable nc = new NotCloseable()) {
    nc.write();
} catch (Exception e) {
    e.printStackTrace();
}

// Fix 3: Remove manual .close() calls and manual finally blocks;
// try-with-resources handles this automatically and safely.
try (FileReader fr1 = new FileReader("a.txt");
    FileReader fr2 = new FileReader("b.txt")) {
    // Resources are automatically closed here
} catch (IOException e) {
    e.printStackTrace();
}
}
}
}

```

The screenshot shows the SIMATS Online Programming Lab interface. The top header includes the SIMATS logo, the text "SIMATS Online Programming Lab", and a user profile for "MATTA SWARNA AMBICA" with ID "192472390RA". Below the header, the "Your Code" section displays the following Java code:

```

1 import java.io.*;
2 public class TryWithResourcesDemo {
3     public static void main(String[] args) {
4         try (BufferedReader reader = new BufferedReader(new FileReader("a.txt"))) {
5             String line = reader.readLine();
6             System.out.println(line);
7         } catch (IOException e) {
8             System.err.println("Error reading data.txt: " + e.getMessage());
9         }
10        try (FileWriter writer = new FileWriter("out.txt")) {
11            writer.write("Hello");
12        } catch (IOException e) {
13            e.printStackTrace();
14        }
15        class NotCloseable implements AutoCloseable {
16            void write() {
17                System.out.println("Writing with NotCloseable");
18            }
19
20            @Override
21            public void close() {
22                System.out.println("NotCloseable closed automatically.");
23            }
24        }
25        try (NotCloseable nc = new NotCloseable()) {

```

The "Input" section shows "stdin input". The "Output" section displays the following text:

```

Writing with NotCloseable
NotCloseable closed
automatically.
le specified)
Error handling files: a.txt (The
system cannot find the file

```

The bottom of the screenshot shows a Windows taskbar with various application icons and a system tray with the date "13-08-202" and time "20:5".

3. Code :

```

public class WaitNotifyDemo {
    private boolean dataReady = false;
    private final Object lock = new Object();

```

```

public void produce() {
    synchronized (lock) {
        dataReady = true;
        System.out.println("Data produced");
        lock.notify();
    }
}

public void consumeCorrectly() {
    synchronized (lock) {
        while (!dataReady) {
            try {
                lock.wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
        System.out.println("Data consumed");
    }
}

public static void main(String[] args) throws InterruptedException {
    WaitNotifyDemo demo = new WaitNotifyDemo();
    Thread consumer = new Thread(demo::consumeCorrectly);
    Thread producer = new Thread(demo::produce);

    consumer.start();
    // Small sleep to ensure consumer reaches the wait state first
    Thread.sleep(100);
    producer.start();

    consumer.join();
    producer.join();
}

```

Your Code

```

1 public class WaitNotifyDemo {
2     private boolean dataReady = false;
3     private final Object lock = new Object();
4
5     public void produce() {
6         synchronized (lock) {
7             dataReady = true;
8             System.out.println("Data produced");
9             lock.notify();
10        }
11    }
12    public void consumeCorrectly() {
13        synchronized (lock) {
14
15            while (!dataReady) {
16                try {
17                    lock.wait();
18                } catch (InterruptedException e) {
19                    Thread.currentThread().interrupt();
20                }
21            }
22            System.out.println("Data consumed");
23        }
24    }
25    public static void main(String[] args) throws InterruptedException

```

Input

stdin input

Output

Data produced
Data consumed